Giảm chi phí API và hạn mức Claude

Claude Code, claude.ai, desktop và mobile

Ghi chú thực hành · Đối chiếu tài liệu Anthropic ngày 27/10/2026

Claude tính tiền theo hai cách. Gọi API thì trả đô la Mỹ theo input và output. Gói Pro, Max, Team và seat-based Enterprise thì trừ vào session limit 5 giờ và weekly limit. Hạn mức đó dùng chung cho chat trên web, desktop, mobile và Claude Code. Với Team và seat-based Enterprise, hạn mức tính riêng từng seat và còn gồm Cowork. Hết hạn mức thì dừng, trừ khi đã bật extra usage. Extra usage tính theo giá API. Làm đúng các mục dưới đây thì giảm cả tiền API lẫn hạn mức bị trừ.

Nguồn: Mỗi lượt gửi lại cả cuộc trò chuyện; Session limit và weekly limit

Quy tắc chính

Output đắt gấp năm lần input. Token thinking cũng tính như output. Chọn model vừa đủ việc, mỗi session chỉ một việc, và chỉ yêu cầu Claude viết dài khi đoạn đó cần giữ lại.

Nguồn: Thinking tính như output

1 Giá API

Mỗi lượt, Claude nhận lại cả cuộc trò chuyện. Ba tên dưới đây là ba thứ khác nhau:

- **cache read** (cache hit): lần sau dùng lại đoạn đã xử lý, nên rẻ hơn.
- **cache write**: lần đầu ghi đoạn đó vào cache. Đắt hơn input, và chỉ trả lúc ghi.
- **Memory**: mục Claude giữ giữa các cuộc trò chuyện trên claude.ai. Không phải giá trong bảng.

Cache read không xóa phần mới, và không làm cuộc trò chuyện ngắn đi. Output vừa viết trong lượt này chưa tính theo giá cache read. Sang lượt sau, phần đó có thể là cache read nếu cache còn hiệu lực.

Claude không đếm theo từ. Claude đếm theo token, rồi tính tiền trên số token. Giá trong bảng là đô la Mỹ cho một triệu token.

Nguồn: Claude Code gửi lại toàn bộ context; cache read và cache write

Bảng 1. Giá niêm yết ngày 27/10/2026. Đơn vị: đô la Mỹ cho một triệu token. 5m write là cache write 5 phút. 1h write là cache write 1 giờ.

Model	Input	Cache read	Output	5m write	1h write
Haiku 5.5, không quá 100.000 token	0,10	0,01	0,50	0,125	0,20
Haiku 5.5, trên 100.000 token	0,50	0,05	2,50	0,625	1
Sonnet 5.5	2	0,10	10	2,50	4
Opus 5.5	4	0,20	20	5	8
Fable 5	10	1	50	12,50	20

Nguồn: Bảng giá model

Ba điểm rút ra từ bảng:

- **Output đắt gấp năm lần input.** Một triệu token output của Sonnet 5.5 là 10 đô la. Một triệu token cache read của Sonnet chỉ 0,10 đô la. Token thinking nằm ở cột output.
- **Chỉ phần lặp lại mới được giá cache read.** Cột 5m write bằng 1,25 lần giá input. Cột 1h write bằng 2 lần. Cache read bằng 5% giá input với Sonnet 5.5 và Opus 5.5, và bằng 10% với Haiku 5.5 và Fable 5. Claude Code tự bật prompt caching. Hết thời hạn cache thì lượt sau phải cache write lại.

Nguồn: Hệ số cache write và cache read

- **Haiku 5.5 có một mốc.** Prompt không quá 100.000 token, kể cả cache read và cache write, vẫn là 0,10 và 0,50

đô la. Vượt mốc thì cả input lẫn output của request đó chuyển sang 0,50 và 2,50 đô la. Request trước đó giữ giá cũ.

Nguồn: Mốc 100.000 token của Haiku 5.5

Fast mode của Opus 5.5 nhân đôi giá input và output, thành 8 đô la và 40 đô la cho một triệu token. Hệ số cache read và cache write tính trên giá input đã nhân đôi. Batch API giảm 50% giá input và output. Hệ số cache nhân tiếp trên giá đã giảm, nên cache read cũng chỉ còn một nửa. Fast mode không dùng được với Batch API.

Nguồn: Fast mode; Batch API giảm 50%

Nguồn: Hệ số nhân trên giá Batch API; Hệ số nhân trên giá fast mode

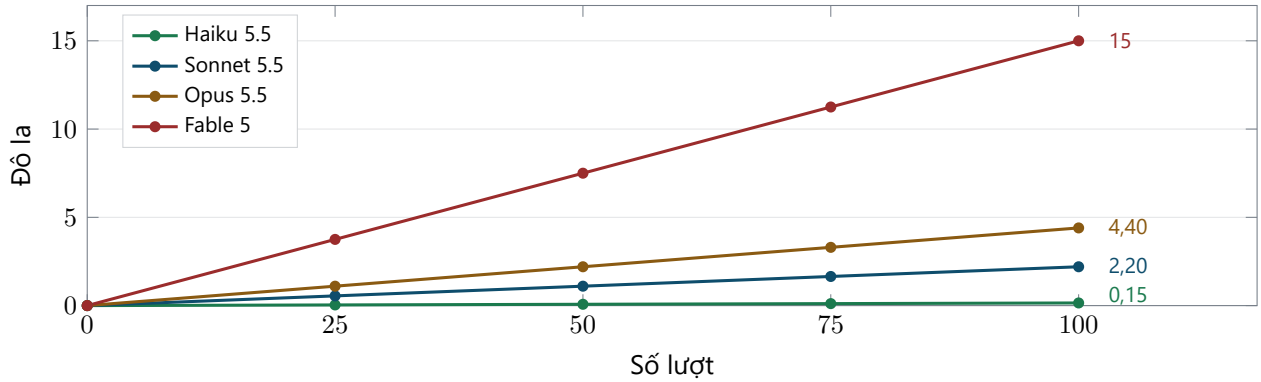
Từ Claude 4.7, và với Claude Mythos Preview, tokenizer đếm cùng một đoạn chữ nhiều hơn khoảng 30% token. Mức tăng tùy nội dung. Sonnet 4.6 trở về trước dùng tokenizer cũ. Giá mỗi token không đổi. Hóa đơn tăng vì số token tăng.

Nguồn: Tokenizer từ Claude 4.7

2 Cùng một việc, năm cách tính tiền

Các hình dưới đây dùng một lượt mẫu, trừ Hình 4. Một lượt mẫu gồm 80.000 token cache read, 2.000 token input mới và 1.000 token output. Cộng lại là 82.000 token input, nên Haiku 5.5 vẫn ở mức giá thấp. Tiền một lượt bằng tổng của ba khoản, rồi chia cho 1.000.000: 80.000 nhân giá cache read, 2.000 nhân giá input, và 1.000 nhân giá output. Nhân với số lượt thì ra tiền cộng dồn. Đây là phép tính từ Bảng 1, không phải hóa đơn thật của một tài khoản.

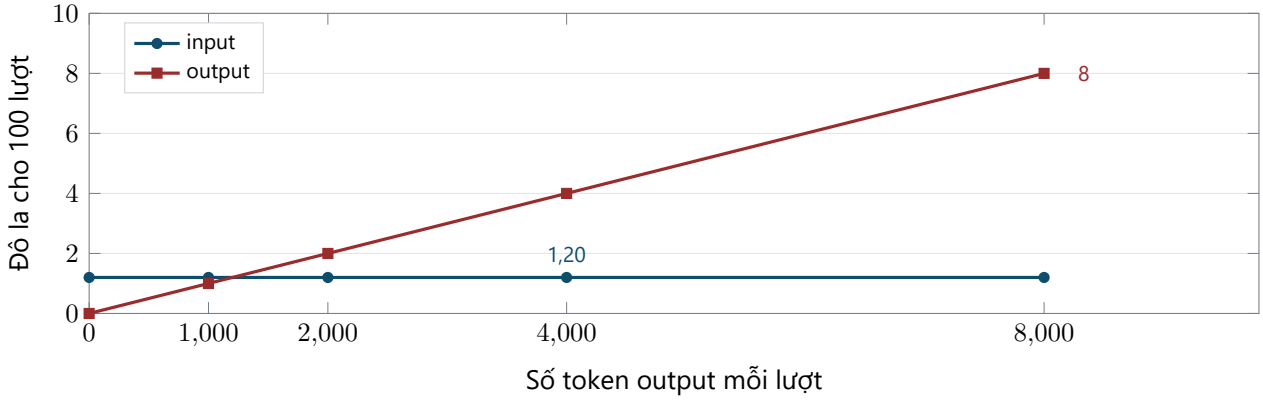
Hình 1. Tiền cộng dồn của bốn model trên cùng một lượt mẫu.



Nguồn: Giá bốn model trong lượt mẫu

Trên lượt mẫu này, Fable 5 tốn gần bảy lần Sonnet 5.5. Haiku 5.5 rẻ đến mức đường nằm sát trục ngang, nên số 0,15 được ghi ở cuối đường.

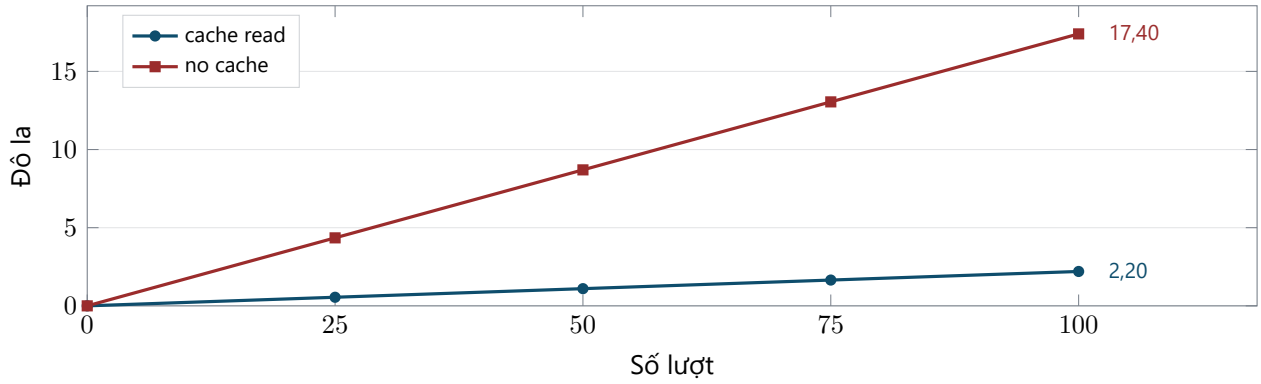
Hình 2. Sonnet 5.5, 100 lượt. Input không đổi. Output tăng theo số token.



Nguồn: Giá input và output của Sonnet 5.5

Tại 1.000 token output, 100 lượt là 2,20 đô la. Tại 8.000 token output, riêng output là 8 đô la, và 100 lượt là 9,20 đô la.

Hình 3. Sonnet 5.5. Đường cache read là lượt mẫu. Đường no cache coi 82.000 token như input mới ở mọi lượt.



Nguồn: Giá cache read so với giá input

Cache write 5 phút cho 80.000 token của Sonnet 5.5 tốn 0,20 đô la. Khoản này chỉ trả một lần và không nằm trên đường cache read. Đổi model thì lượt sau bị cache miss, dù nội dung không đổi. Bật hoặc tắt fast mode cũng vậy, vì fast mode có cache riêng.

Nguồn: Đổi model

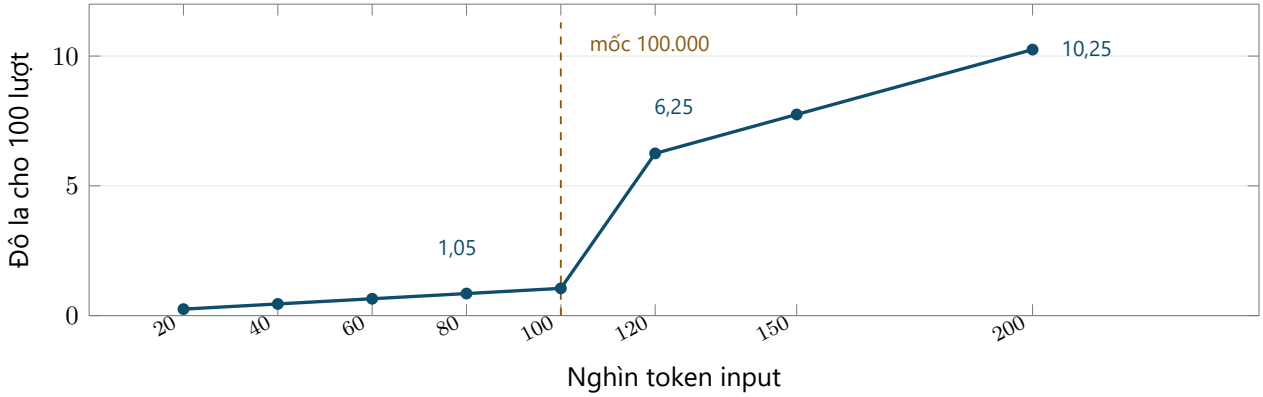
Đổi effort thì cache miss, trừ Opus 5.5, Sonnet 5.5, Haiku 5.5 và Fable 5.1 khi gọi bằng API key hoặc dùng subscription Claude. Fable 5 thì cache miss. Việc giữ cache này không áp dụng trên Amazon Bedrock, Google Cloud's Agent Platform, Claude apps gateway và tài khoản HIPAA. Nếu đặt `CLAUDE_CODE_DISABLE_EXPERIMENTAL_BETAS` thì cache cũng không được giữ.

Nguồn: Đổi effort; Trường hợp không giữ cache

Bật hoặc tắt skill, command hoặc theme thì không làm cache miss. Plugin có MCP server thì theo quy tắc của MCP server đó. Deny cả một tool chỉ làm cache miss khi tool search đang tắt, tức danh sách tool được nạp từ đầu. Khi tool search đang bật (mặc định), mô tả tool chỉ gửi lúc cần, và cache vẫn còn. Nếu đoạn mở đầu giữ nguyên, các lượt sau lại là cache read. Đường no cache là trường hợp mọi lượt đều tính như input mới.

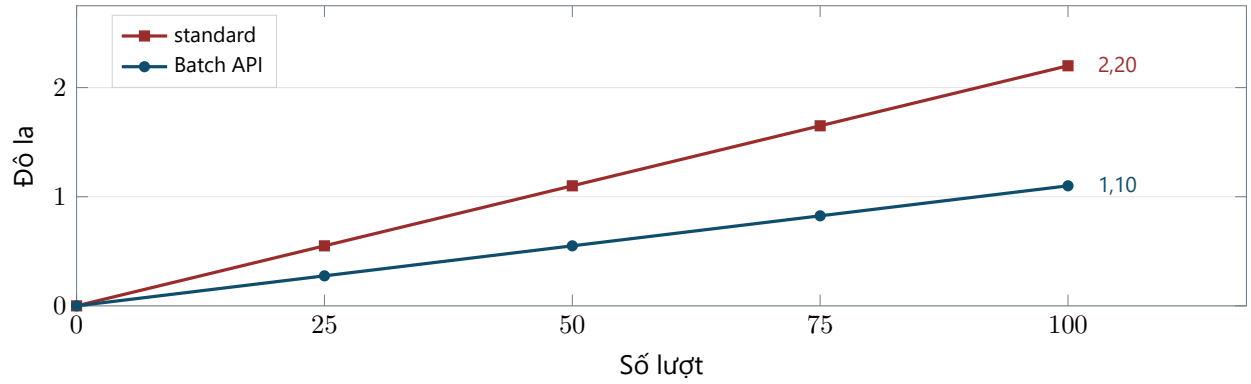
Nguồn: Bật hoặc tắt skill

Hình 4. Haiku 5.5, 100 lượt, 1.000 token output, no cache. Vượt 100.000 token input thì cả input lẫn output chuyển sang giá cao.



Nguồn: Mốc giá Haiku 5.5

Hình 5. Sonnet 5.5 trên lượt mẫu: giá standard và Batch API.



Nguồn: Giá Batch API của Sonnet 5.5

Opus 5.5 ở fast mode dùng bảng giá khác: 8 đô la cho một triệu token input và 40 đô la cho một triệu token output, và không dùng được với Batch API. Trong hạn mức gói, Pro, Max, Team và seat-based Enterprise không có hóa đơn từng lượt. Model mạnh, output dài và cache miss vẫn làm hạn mức giảm nhanh hơn. Bật extra usage thì tính đúng các mức đô la ở trên.

Nguồn: Giá fast mode của Opus 5.5

3 Hạn mức của gói

Pro và Max có session limit, làm mới sau 5 giờ, và có weekly limit. Chat trên web, desktop, mobile và Claude Code trong terminal, VS Code hoặc IDE được hỗ trợ đều trừ chung một hạn mức.

Nguồn: Pro và Max dùng chung một hạn mức

Hạn mức hết sớm khi tin nhắn dài, cuộc trò chuyện dài, file đính kèm lớn, model mạnh, effort cao, hoặc khi Claude Code làm trên repo lớn và tự chạy nhiều lệnh. Số tin nhắn gửi được không cố định. Xem tại `claude.ai/settings/usage`. Trong Claude Code, lệnh `/usage` cho biết phần nào đang chiếm hạn mức: skill, subagent, plugin và MCP server. Phần này tính từ máy đang mở, không gồm máy khác hay claude.ai. Thanh hạn mức của gói thì tính trên cả tài khoản. Với tài khoản API, `/usage` hiện số đô la ước tính. Số cạnh spinner là ước lượng token output của lượt này, gồm chữ Claude viết, tool call, thinking và subagent chạy ngay trong session. Số đó không gồm input.

Nguồn: Số tin không cố định; Số cạnh spinner

Trên Max, trên premium seat của Team và trên premium seat của seat-based Enterprise, Fable 5 và Fable 5.1 dùng tối đa một nửa weekly limit. Hết phần đó thì Fable chạy bằng usage credits, hoặc đổi sang model khác để tiếp tục trong hạn mức gói. Trên Pro và trên standard seat của Team, hai model này không nằm trong hạn mức gói, chỉ chạy bằng usage credits. Standard seat của seat-based Enterprise cũng vậy, nhưng chỉ khi tổ chức đã bật usage credits.

Nguồn: Một nửa weekly limit; Pro và standard seat

Nguồn: Standard seat của Enterprise

Team và seat-based Enterprise tính hạn mức theo từng seat. Hạn mức đó dùng chung cho chat, Claude Code và Cowork. Hết hạn mức của seat thì chờ đến lúc làm mới, hoặc dùng tiếp nếu đã bật extra usage. Usage-based Enterprise không có hạn mức từng seat, mà tính theo giá API.

Nguồn: Per-seat allowance; Usage-based Enterprise

4 Năm cách giảm chi phí nhiều nhất

1. **Hạ model ngay khi mở session.** Sonnet 5.5 cho hầu hết việc viết mã. Haiku 5.5 cho đổi tên, regex, lọc log và subagent làm việc lặp. Opus 5.5 khi việc khó hoặc phải sửa nhiều phần. Fable 5 chỉ khi việc xứng với số tiền đó. Nhiều gói mặc định model mạnh. Đổi model giữa session thì cache miss.

Nguồn: Chọn model; Đổi model thì cache miss

2. **Bớt output.** Hạ effort bằng lệnh `/effort`. Opus 5.5, Sonnet 5.5, Haiku 5.5 và Fable luôn có thinking, không tắt được. Model cũ hơn, nếu vẫn tắt được, thì tắt khi không cần. Yêu cầu Claude chỉ nêu tên file đã sửa và lệnh kiểm tra, không viết lại cả file. Không bật fast mode nếu muốn tiết kiệm.

Nguồn: Không tắt được thinking

3. **Một session, một việc.** Đổi việc thì `/clear`. `/clear` không tốn một lượt tóm tắt. Cùng một việc mà cuộc trò chuyện đã dài thì `/compact`, và nói rõ phần cần giữ: file đã sửa, lệnh kiểm tra, lỗi còn mở. Khi cache còn, `/compact` chủ yếu trả tiền cho bản tóm tắt. Khi cache đã hết thời hạn, lượt đó phải xử lý lại cả cuộc trò chuyện. Câu hỏi phụ dùng `/btw`. Câu đó không vào history, nên các lượt sau không phải trả tiền cho nó. Chính câu hỏi phụ vẫn tốn một lượt nhỏ.

Nguồn: `/clear` không tốn lượt tóm tắt; `/btw` không vào history

4. **Đừng đưa file thừa vào cuộc trò chuyện.** Dán cả file, hoặc `@` một file lớn, là giữ nguyên nội dung đến cuối session. Nêu đường dẫn và tên hàm. Log dài thì giao subagent, và chỉ đưa phần lỗi về session chính. Subagent vẫn tốn tiền. Trong file cấu hình, ghi `model: haiku`. Chỉ dùng `/fork` khi cần một session nền giữ nguyên history.

Nguồn: Subagent chỉ trả phần tóm tắt; `model: haiku`

5. **Tắt việc tự chạy.** `/loop` và scheduled task trong session gửi lại cả cuộc trò chuyện theo chu kỳ, kể cả lúc không ngồi máy. Goal check-in hỏi lại khi máy rảnh, tối đa ba lần giữa hai câu của bạn, và mỗi lần gửi lại cả cuộc trò chuyện. Tắt bằng `CLAUDE_CODE_GOAL_CHECKIN_MINUTES=0`. Vài session chạy song song trừ chung một hạn mức gói. Dùng API thì mỗi session là một hóa đơn riêng.

Nguồn: Scheduled task và goal check-in

5 Claude Code đưa gì vào ngay từ câu đầu

Những mục sau có mặt từ lượt đầu và được gửi lại ở mọi lượt:

- `CLAUDE.md` nên dưới 200 dòng. Chỉ ghi lệnh kiểm tra, chỗ dễ hiểu nhầm, và việc đã phải sửa lại hai lần. Tách file nhưng vẫn nạp hết lúc mở session thì không giảm tiền. Việc ít dùng, như rà soát hoặc chuyển hệ thống, để trong skill. Lúc mở session, Claude chỉ thấy description của skill. Nội dung đầy đủ chỉ vào khi skill được gọi. Sau `/compact`, `CLAUDE.md` ở thư mục gốc, rule không có `paths`, auto memory, plan file và skill description được nạp lại. Nội dung skill đã gọi được giữ lại, nhưng mỗi skill chỉ giữ 5.000 token đầu, và tất cả cộng lại không quá 25.000 token.

Nguồn: `CLAUDE.md` dưới 200 dòng; Chỉ nạp skill description

Nguồn: Giới hạn nội dung skill sau `/compact`

- Tắt MCP server không dùng bằng `/mcp`. Đã có lệnh `gh` hoặc `aws` thì gọi lệnh đó, không thêm MCP server. Chỉ tên tool và instruction của server vào context. Mô tả đầy đủ chỉ gửi khi dùng đúng tool đó.

Nguồn: Tên tool vào trước, mô tả đầy đủ khi dùng

- Output style Explanatory thêm khối Insight vào câu trả lời. Output style Learning cũng vậy, và còn chừa một đoạn mã để người dùng tự viết. Những khối đó nằm trong cuộc trò chuyện và đi theo các lượt sau. Bật khi đang học. Tắt khi đang làm việc.

Nguồn: Explanatory và Learning

- Plan mode, phím `Shift+Tab`, dùng trước khi sửa nhiều file. Claude đọc repo và đưa một trang kế hoạch để duyệt trước khi sửa. Làm vậy tránh nhiều vòng sửa sai trên model đắt.

Nguồn: Plan mode

`/context` cho biết mục nào làm đầy context window. `/usage` cho biết tiền hoặc hạn mức đang vào đâu, và cache còn hay đã hết thời hạn. Bảng phân bổ trong `/usage` trên CLI không gồm máy khác hay claude.ai. Thanh hạn mức của gói thì lấy từ tài khoản.

Nguồn: Phân bổ trên máy này

Cache miss khi đổi model hoặc bật fast mode. Đổi effort chỉ làm cache miss với model không nằm trong nhóm của Hình 3. Fable 5 thuộc nhóm bị cache miss. Bật hoặc tắt skill, command hoặc theme thì không mất. Plugin có MCP server thì theo quy tắc của MCP server đó. Deny cả một tool chỉ làm cache miss khi tool search đang tắt. Nghỉ quá thời hạn cache rồi hỏi tiếp một session dài thì phải cache write lại. Path-scoped rule trong `.claude/rules/` chỉ nạp khi chạm file đó. Rule không có `paths` thì nạp từ đầu.

Nguồn: Path-scoped rule

Khi chạy với `-p`, `--max-budget-usd` dừng session khi tiền ước tính chạm mức đã đặt, và `--max-turns` dừng khi hết số lượt. Claude Code trên web tính cùng cách, nhưng làm trên checkout đã đưa lên server, không phải checkout trên máy. Session web không thấy file chưa push, cũng không thấy kết nối chỉ cài trên máy. Session trên máy và session web không dùng chung cuộc trò chuyện. Đừng làm cùng một việc ở cả hai nơi.

Nguồn: `--max-budget-usd`; Session web không thấy checkout máy

6 Chat trên web, desktop và mobile

Chat trên web dùng cùng hạn mức với Claude Code. Hạn mức hết nhanh vì tin dài, file lớn, Research, web search, model mạnh và Artifacts.

- Đổi chủ đề thì mở cuộc trò chuyện mới.
- File cần hỏi nhiều lần thì đưa vào Projects một lần. Khi còn trong cache, phần đó trừ hạn mức ít hơn so với tải lên lại. Sau một thời gian không dùng, lần quay lại đầu tiên tính như nội dung mới. Project instructions áp dụng cho mọi chat trong project, nên viết ngắn. Các chat trong một project dùng chung knowledge base, không dùng chung history.

Nguồn: Nội dung Projects được cache; Project instructions

- Khi knowledge base gần đầy context window, gói Pro, Max, Team và Enterprise tự chuyển sang RAG. Claude chỉ lấy đoạn liên quan, không nạp cả kho. Lúc đó project chứa được khoảng mười lần context window. File không còn đúng thì gỡ ra, vì lần tìm sau vẫn thấy file đó.

Nguồn: RAG trên gói trả phí; Kho rộng khoảng mười lần

Nguồn: Chỉ lấy đoạn liên quan

- Research và web search là thêm tool call, nên trừ thêm hạn mức. Với Artifacts, sửa một đoạn, không viết lại cả tài liệu.

Nguồn: Research và web search

- Sửa mã thì chuyển sang Claude Code. Không đổ cả repo vào Projects.
- Memory bật sẵn ở Free, Pro và Max, trên web, desktop và mobile. Trên Team và Enterprise, owner bật cho tổ chức trước. Từng người vẫn tắt cho đến khi tự bật. Xóa mục Memory nào không còn đúng.

Nguồn: Memory bật sẵn

Khi hết hạn mức

Extra usage không còn nằm trong giá gói. Nó tính như API: model mạnh, output dài và session dài đều ra đô la. Xem mục usage trên claude.ai trước khi bật extra usage.

7 Khi tự gọi API

Claude Code đã tự đặt cache. Ứng dụng của bạn phải tự đặt:

- Phần không đổi, gồm system prompt, tài liệu và ví dụ, để trước. Phần đổi mỗi lượt để sau. Cache chỉ khớp khi đoạn mở đầu giống hệt.
- Đặt cache bằng trường `cache_control` ở đầu request. Claude tự đặt breakpoint, tức mốc cache, khi cuộc trò chuyện dài ra. Khi cần chọn từng đoạn, đặt `cache_control` trên đúng block đó.

Nguồn: `cache_control`

- Việc không cần trả lời ngay thì dùng Batch API để giảm 50%.

Nguồn: Batch API giảm 50%

- Prompt gửi cho Haiku 5.5 giữ không quá 100.000 token, kể cả cache read và cache write, nếu muốn giá 0,10 và 0,50 đô la.

Nguồn: Kể cả cache read và cache write

- Hạ effort. Token thinking tính như output, và đó là phần đắt.

- `inference_geo: us` cộng 10% lên mọi loại token với Claude 4.6 trở đi, gồm cả cache read, cache write và mức giá cao của Haiku 5.5. Giữ global (mặc định) thì không bị cộng 10% này.

Nguồn: `inference_geo: us` cộng 10%

8 Việc thường gặp

Bảng 2. Cách làm theo từng việc. Tên model lấy từ Bảng 1.

Việc đang làm	Cách làm ít tốn hơn
Viết mã hằng ngày	Sonnet 5.5, effort vừa hoặc thấp
Đổi tên, lọc log, regex	Haiku 5.5, không quá 100.000 token
Việc khó, phải sửa nhiều phần	Opus 5.5, xong thì trở lại Sonnet 5.5
Đổi sang việc khác	`/clear`
Cùng một việc, session đã dài	`/compact`, nêu phần cần giữ
Hỏi một chi tiết vừa đọc	`/btw`
Sắp sửa nhiều file	Plan mode
Lệnh in ra hàng nghìn dòng	subagent Haiku 5.5, chỉ đưa lỗi về
Hỏi lại một PDF trên web	Đưa vào Projects, mỗi câu một chat mới
Việc gọi API không cần kết quả ngay	Batch API, giảm một nửa

9 Nguồn

Mỗi mục phía trên đã có link tới đúng câu được lấy. Danh sách này là các trang gốc, và cũng mở đúng mục chính.

- Giá model, tokenizer từ Claude 4.7.
  platform.claude.com/docs/en/about-claude/pricing#model-pricing
- cache read, cache write và `cache_control`.
  platform.claude.com/docs/en/about-claude/pricing#prompt-caching
- Mốc 100.000 token của Haiku 5.5.
  platform.claude.com/docs/en/about-claude/pricing#long-context-pricing
- Fast mode và Batch API.
  platform.claude.com/docs/en/about-claude/pricing#fast-mode-pricing
- `inference_geo: us`.
  platform.claude.com/docs/en/about-claude/pricing#data-residency-pricing
- Chi phí Claude Code, `/usage`, `/clear`, `/compact`.
  code.claude.com/docs/en/costs#why-usage-climbs-in-a-long-session
- Việc nào làm cache miss.
  code.claude.com/docs/en/prompt-caching#actions-that-invalidate-the-cache
- Hạn mức Pro và Max.
  support.claude.com/.../use-claude-code-with-your-pro-or-max-plan
- Fable trên từng gói.
  support.claude.com/.../claude-fable-models-on-your-plan
- Hạn mức gói và Projects.
  support.claude.com/.../usage-limit-best-practices